Data Collection and Analysis Using SQL

Presented by: Hamidov Jeyhun

SQL-Based Analysis with Valentina Studio & PostgreSQL

Introduction

In this presentation, I will show how I collected and analyzed data using SQL.

You will learn about:

1. How I gathered the data
2. How I created data tables in SQL and imported the dataset
3. How I analyzed the data in SQL and drew meaningful conclusions

Without further delay, let’s get started!

Data Preparation

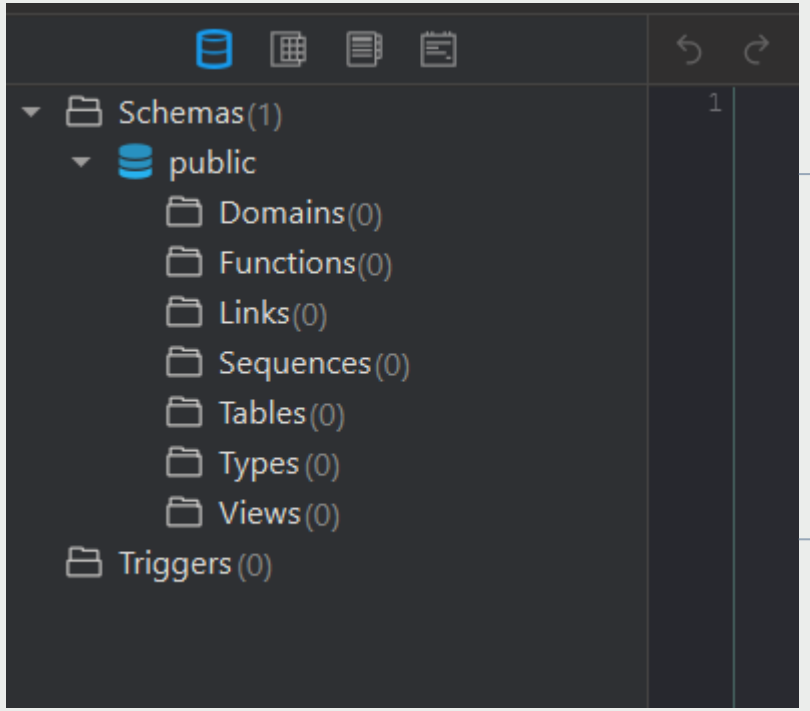
order_items	10/23/2025 10:54 PM	CSV File	111 KB
staffs	10/23/2025 10:54 PM	CSV File	1 KB
products	10/23/2025 10:54 PM	CSV File	17 KB
orders	10/23/2025 10:54 PM	CSV File	76 KB
stocks	10/23/2025 10:54 PM	CSV File	9 KB
customers	10/23/2025 10:54 PM	CSV File	125 KB
brands	10/23/2025 10:54 PM	CSV File	1 KB
stores	10/23/2025 10:54 PM	CSV File	1 KB
categories	10/20/2025 11:52 AM	CSV File	1 KB

In this project, we already have a certain dataset.

However, since all of the data is in **CSV format**, it cannot be directly analyzed in SQL.

Therefore, we need to **import this data into the Valentina Studio program** to make it ready for analysis.

Creating Tables



As you can see here, there are no tables yet.

Therefore, we need to **write the SQL code for each table separately** and create them one by one.

After creating all the tables, we will be able to **perform our analyses on these tables**.

Let’s start to create all tables one by one

Order Items table

```sql
CREATE TABLE public.order_items(
    order_id INT,
    item_id INT,
    product_id INT,
    quantity INT,
    list_price FLOAT,
    discount FLOAT,
    PRIMARY KEY (order_id, item_id, product_id)
)
```

The **Order_Items** table stores detailed information about each product included in a customer order. It connects orders with specific products and contains key details such as the quantity ordered, list price, and applied discount. This table is essential for calculating total sales, discounts, and product-level revenue during data analysis.

order_id → Unique ID of each order

item_id → Unique ID of each item within the order

product_id → Identifier of the product being ordered

quantity → Number of units ordered for each product

list_price → Original price of the product before discount

discount → Discount rate applied to the product in that order

Staffs Table

```sql
CREATE TABLE staffs(
    staff_id INT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(50),
    phone VARCHAR(50),
    active NUM CHECK (active IN (0, 1)),
    store_id INT,
    manager_id INT,
    PRIMARY KEY (staff_id, store_id, manager_id)
)
```

The **Staffs** table contains information about all employees working in the stores. It includes their personal details such as name, email, and phone number, as well as organizational data like store location, manager, and activity status. This table helps track employee performance and manage staff assignments across different stores.

staff_id → Unique ID of each staff member

first_name → Staff member's first name

last_name → Staff member's last name

email → Contact email address of the staff

phone → Staff member's phone number

store_id → The store where the staff member works

active → Indicates whether the staff member is currently active or not

manager_id → ID of the manager supervising the staff member

Orders

```sql
CREATE TABLE orders(
    order_id INT,
    customer_id INT,
    order_status date,
    required_date date,
    shipped_date date,
    store_id INT,
    staff_id INT,
    PRIMARY KEY (order_id, customer_id, store_id, staff_id)
)
```

The **Orders** table keeps records of all customer orders made in the system.It includes details such as order dates, shipment information, store location, and the staff member who processed each order.This table is essential for analyzing sales performance, tracking delivery progress, and understanding customer purchasing behavior.

order_id → Unique identifier for each order

customer_id → ID of the customer who placed the order

order_status → Current status of the order (e.g., pending, shipped, delivered)

required_date → The date by which the customer expects to receive the order

shipped_date → The actual date when the order was shipped

store_id → The store from which the order was processed

staff_id → The staff member responsible for handling the order

Stocks

```sql
CREATE TABLE stocks (
    store_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY(store_id, pro)
)
```

store_id → Identifier of the store where the product is kept

product_id → Identifier of the product in stock

quantity → Number of units of that product available in the store

The **Stocks** table contains information about the quantity of each product available in every store. It helps track inventory levels, identify low-stock products, and ensure that stores are properly supplied. This table is essential for managing stock levels and maintaining smooth product availability across all store locations.

Categories

```sql
CREATE TABLE categories(
    category_id INT PRIMARY KEY,
    category_name VARCHAR(50)
)
```

category_id → Unique identifier for each product category

category_name → Name of the product category

The **Categories** table stores information about different product categories, such as electronics, clothing, or accessories. It helps organize products into groups, making it easier to analyze sales performance by category and understand which types of products perform best.

```sql
CREATE TABLE customers(
    customer_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(50),
    email VARCHAR(50),
    street VARCHAR(50),
    city VARCHAR(50),
    state VARCHAR(50),
    zip_code VARCHAR(50)
)
```

The **Customers** table stores detailed information about all customers, including their personal and contact details as well as address information. It allows businesses to analyze customer demographics, track orders, and identify key markets or locations where sales are strongest.

Customers

customer_id → Unique identifier for each customer

first_name → Customer's first name

last_name → Customer's last name

phone → Customer's contact phone number

email → Customer's email address

street → Customer's street address

city → City where the customer lives

state → State or region of the customer

zip_code → Postal code of the customer's address

```sql
CREATE TABLE brands(
    brand_id INT,
    brand_name VARCHAR(50)
)
```

The **Brands** table stores information about different product brands available in the system. It helps organize and categorize products by brand, making it easier to analyze brand performance, compare sales between brands, and understand customer preferences.

Brands

brand_id → Unique identifier for each brand

brand_name → Name of the brand

Products

```sql
CREATE TABLE products(
    product_id INT,
    product_name VARCHAR(50),
    brand_id INT,
    category_id INT,
    model_year INT,
    list prise FLOAT
    PRIMARY KEY (product_id, brand_id,category_id)
)
```

The **Products** table contains detailed information about all products, including their names, brands, categories, model years, and prices. It serves as a key source for product-related analysis such as pricing trends, brand performance, and category-based sales comparisons.

product_id → Unique identifier for each product

product_name → Name of the product

brand_id → Identifier of the brand the product belongs to

category_id → Identifier of the category of the product

model_year → Year the product model was released

list_price → Standard listed price of the product

Store

```sql
CREATE TABLE store(
    store_id INT PRIMARY KEY,
    store_name VARCHAR(50),
    phone VARCHAR(50),
    email VARCHAR(50),
    street VARCHAR(50),
    cty VARCHAR(50),
    state VARCHAR(50),
    zip_code VARCHAR(50)
)
```

store_id → Unique identifier for each store

store_name → Name of the store

phone → Contact phone number of the store

email → Store's official email address

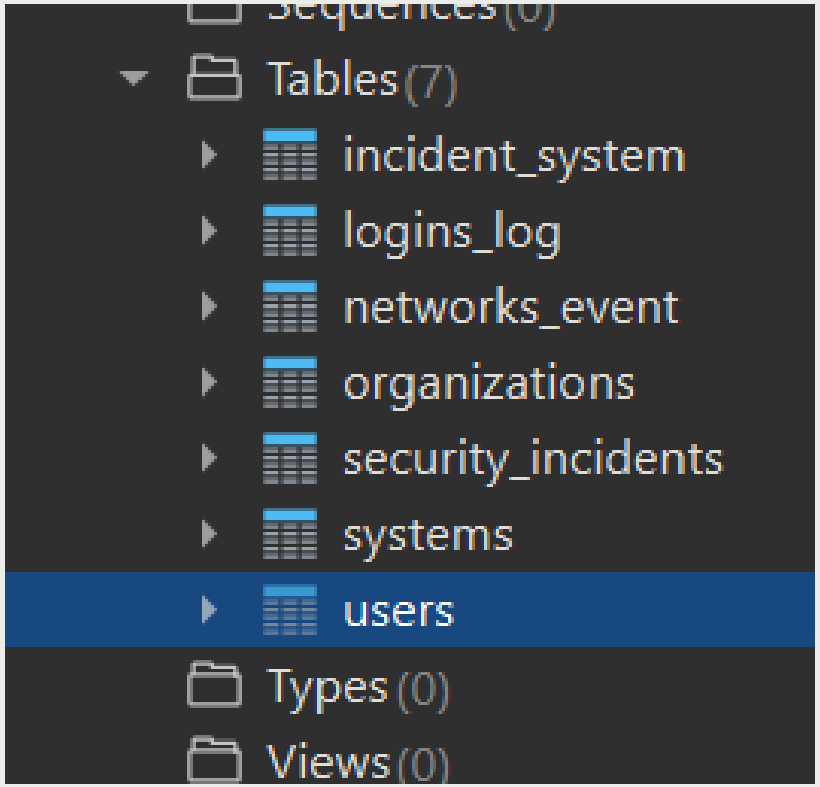
street → Street address of the store

city → **City where the store is located**

state → **State or region of the store**

zip_code → Postal code of the store's address

The **Stores** table stores information about all store locations, including their contact details and addresses. It helps manage store operations, track sales by location, and analyze performance across different regions.



Database Setup Summary

In this part of the project, we successfully collected the dataset and organized it into a SQL database. Each table was created with clearly defined columns, allowing us to build strong relationships between customers, orders, products, and stores. This structure makes it easier to perform accurate data analysis and extract valuable business insights in the next phase.

Dataset Overview

The dataset used in this project represents a complete sales system that connects customers, orders, products, brands, categories, stores, and staff. It contains multiple related tables, which together form a realistic business database. Each table provides a different perspective — from customer information and order details to stock levels and store performance.

NOW LET’S START TO ANALYZE THIS DATA

Total Revenue and Number of Orders

```sql
SELECT
    count(DISTINCT o.order_id) AS total_orders,
    round(sum(oi.quantity * oi.list_price * (1-oi.discount))::NUMERIC, 0) AS total_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
```

	total_orders	total_revenue
1	1615	7689117

In this part of the analysis, we looked at the company's general sales results. We calculated how many orders were made and how much total money the company earned. The results show that the company had **1,615 orders** and made about **7,689,117** in total revenue. These numbers show that sales are strong and that the company has a good number of loyal customers. It also means that the prices and sales methods are working well. Overall, this analysis gives a simple and clear view of how the company is doing financially and helps us understand the business performance better for the next parts of the report.

Top 10 Best Selling Products

```sql
SELECT
    p.product_name,
    round(sum(oi.quantity * oi.list_price * (1 - oi.discount)):: NUMERIC, 0) AS revenue
FROM order_items AS oi
JOIN products AS p ON oi.product_id = p.product_id
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 10
```

	product_name	revenue
1	Trek Slash 8 27.5 - 2016	555559
2	Trek Conduit+ - 2016	389249
3	Trek Fuel EX 8 29 - 2016	368473
4	Surly Straggler 650b - 2016	226766
5	Trek Domane SLR 6 Disc - 2017	211585
6	Surly Straggler - 2016	203508
7	Trek Remedy 29 Carbon Frameset - 2016	203381
8	Trek Powerfly 8 FS Plus - 2017	188250
9	Trek Madone 9.2 - 2017	175900
10	Trek Silque SLR 8 Women's - 2017	174525

In this part of the analysis, we looked at the **top 10 best-selling products**. Overall, the **Trek** brand achieved the highest sales among all companies. However, when we look at individual products, the **Trek Slash 8 27.5 – 2016** had the best sales results, showing that it is a very successful product. After that, we can see **Trek Conduit+ – 2016**, **Trek Fuel EX 8 29 – 2016**, and a product from another brand, **Surly Straggler 650b – 2016**, also performing well. These products are all among the **top five**. In general, this analysis helps us understand which products are more successful and which ones the company should **focus on more** in the future.

Sales by Category

```sql
SELECT
    c.category_name,
    round(sum(oi.quantity * oi.list_price * (1 - oi.discount))::NUMERIC, 0) AS
total_sales
FROM order_items AS oi
JOIN products AS p ON oi.product_id = p.product_id
JOIN categories AS c ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_sales DESC
```

From this analysis, we can see that each category has a certain level of sales, but they all differ from each other to some extent. Looking at the first place, the **Mountain Bikes** category has the highest sales. This category achieved a total of **2,715,080** in sales, clearly showing how much stronger it is compared to the others.
In the following positions, we can see **Road Bikes**, **Cruisers Bicycles**, and **Electric Bikes**. Each of these categories has sales of over **900,000**, which indicates solid performance as well. Overall, this analysis helps us understand **which categories we should focus on more** and **where we can develop new and different products** to increase future sales.

	category_name	total_sales
1	Mountain Bikes	2715080
2	Road Bikes	1665098
3	Cruisers Bicycles	995033
4	Electric Bikes	916685
5	Cyclocross Bicycles	711012
6	Comfort Bicycles	394020
7	Children Bicycles	292189

Sales by Brand

```sql
SELECT
    b.brand_name,
    round(sum(oi.quantity * oi.list_price * (1 - oi.discount))::NUMERIC, 0) AS revenue
FROM order_items AS oi
JOIN products AS p ON oi.product_id = p.product_id
JOIN brands AS b ON p.brand_id = b.brand_id
GROUP BY b.brand_id
ORDER BY revenue DESC
```

Even though some results were visible in the first analysis, it is still important to review the brands separately. **Trek** had many high-revenue products, and **Electra** has now moved up to second place, while **Surly** dropped to third. However, **Trek** and **Electra** lead mainly because they have more products, while **Surly** achieved impressive results despite having four to five times fewer items.
In conclusion, **Surly deserves more focus**, as increasing its product range could easily help it reach the same level of success as **Electra**, and possibly even get close to **Trek**.

	brand_name	revenue
1	Trek	4602754
2	Electra	1205321
3	Surly	949507
4	Sun Bicycles	341995
5	Haro	185385
6	Heller	171459
7	Pure Cycles	149476
8	Ritchey	78899
9	Strider	4320

Quantity of products in every brand

```sql
SELECT
    b.brand_name AS brands,
    count(p.product_id) AS total_products_quantity
FROM brands AS b
JOIN products AS p ON b.brand_id = p.brand_id
GROUP BY b.brand_name
ORDER BY total_products_quantity DESC
```

From this analysis, it is clear that **Trek** and **Electra** have more products than other brands. However, **Surly** still showed strong performance in total revenue.
The main reason Trek and Electra lead in revenue is their larger product range, while Surly achieved great results despite having 4–5 times fewer products.
In conclusion, **Surly deserves more attention**, and if it increases its product count, it could reach the same success level as Electra — maybe even get close to Trek.

	brands	total_products_quantity
1	Trek	135
2	Electra	118
3	Surly	25
4	Sun Bicycles	23
5	Haro	10
6	Pure Cycles	3
7	Heller	3
8	Strider	3
9	Ritchey	1

Top Performing Staff

```sql
SELECT
    Concat(s.first_name, ' ', s.last_name) AS staff_name,
    round(sum(oi.quantity * oi.list_price *(1 - oi.discount))::NUMERIC, 0) AS total_sales
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN staffs AS s ON o.staff_id = s.staff_id
GROUP BY staff_name
ORDER BY total_sales DESC
```

In this section, we analyzed the **top-performing staff members** and came across some surprising results. **Marcelene Boyer** and **Venita Daniel** showed outstanding dominance, each achieving over **2,600,000** in total sales.
In contrast, the next staff members — **Genna Serrano** and **Mireya Copeland** — had sales around **750,000**, which is significantly lower compared to the two leaders.

	staff_name	total_sales
1	Marcelene Boyer	2624121
2	Venita Daniel	2591631
3	Genna Serrano	853287
4	Mireya Copeland	752536
5	Kali Vargas	463918
6	Layla Terrell	403624

Top 10 cities by their revenue

```sql
SELECT
    c.city,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount))::NUMERIC, 0) AS total_revenue
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN customers AS c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC
LIMIT 10
```

In this part of the analysis, we looked at the **total revenue by city**. As shown in the table, **Mount Vernon** ranks first with a total revenue of **105,563**. It is followed by **Ballston Spa** and **San Angelo**, both having around **98,000** in total revenue. Overall, the top five cities have very close results, which shows that sales are evenly distributed across these regions. Cities like **Houston** and **Astoria** are at the lower end, but they still maintain a stable sales level. This analysis shows that future sales strategies should **focus more on cities like Mount Vernon, Ballston Spa, and San Angelo**, while additional promotions or campaigns could help **boost sales in the lower-performing areas**.

	city	total_revenue
1	Mount Vernon	105563
2	Ballston Spa	98620
3	San Angelo	98429
4	Baldwinsville	96376
5	Howard Beach	95329
6	Orchard Park	90920
7	Canyon Country	86521
8	Monroe	84076
9	Houston	81022
10	Astoria	79824

Top 10 customers with high purchases

```sql
SELECT
    concat(c.first_name, ' ', c.last_name) AS customer_name,
    count (o.order_id) AS total_orders,
    round(sum(oi.quantity * oi.list_price * (1 - oi.discount))::NUMERIC, 0) AS
total_spent
FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
JOIN order_items AS oi ON o.order_id = oi.order_id
GROUP BY customer_name
ORDER BY total_spent DESC
LIMIT 10
```

In this part of the analysis, we focused on the **top customers based on their total orders and spending**. The data clearly shows that **Sharyn Hopkins** is the leading customer with **10 orders** and a total spending of **34,808**. She is followed closely by **Pamela Newman** and **Abby Gamble**, both showing strong purchase activity with more than **32,000** in total spent. Overall, we can see that the top five customers have similar spending levels, all above **31,000**, which indicates a consistent group of loyal buyers. However, the lower part of the list shows a noticeable drop in spending, meaning that there is potential to **increase engagement and sales among less active customers** through personalized offers or loyalty programs.

	customer_name	total_orders	total_spent
1	Sharyn Hopkins	10	34808
2	Pamelia Newman	11	33634
3	Abby Gamble	7	32803
4	Lyndsey Bean	11	32675
5	Emmitt Sanchez	12	31926
6	Melanie Hayes	7	31914
7	Debra Burks	11	27888
8	Elinore Aguilar	10	25636
9	Corrina Sawyer	8	25613
10	Shena Carter	4	24891

Impact of Discount Rates on Total Revenue

```sql
SELECT
    ROUND((oi.discount * 100)::NUMERIC, 0) AS discount_percent,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount))::NUMERIC, 0) AS
total_revenue
FROM order_items AS oi
GROUP BY discount_percent
ORDER BY discount_percent
```

In this part of the analysis, we examined the **impact of discounts on total revenue**. The results show that **lower discounts** lead to **higher total revenue**. For example, a **5% discount** generated the highest revenue of **2,138,329**, while the **20% discount** resulted in the lowest revenue of **1,679,166**. This clearly shows that giving **larger discounts** does not always increase total sales income. Instead, **smaller discounts** help maintain profitability while still encouraging customer purchases.

	discount_percent	total_revenue
1	5	2138329
2	7	2026738
3	10	1844883
4	20	1679166

Overall Conclusion

In this project, we analyzed different parts of the company’s sales data, including brands, products, categories, staff, customers, cities, and discounts. The results clearly show that **Trek** and **Electra** are the top-performing brands, while **Surly**, even with fewer products, showed very strong results in total revenue.
Among the staff, **Marcelene Boyer** and **Venita Daniel** had the highest sales, showing their strong performance compared to others. The **top customers**, such as **Sharyn Hopkins** and **Pamela Newman**, also played an important role in the company’s total income.
Looking at cities, **Mount Vernon** had the highest revenue, while other cities like **Ballston Spa** and **San Angelo** also performed well. The discount analysis showed that **smaller discounts (like 5%)** bring **higher total revenue**, which means large discounts do not always help the company earn more.
Overall, this analysis gives a clear view of the company’s performance and helps to understand which areas need more focus — for example, increasing Surly’s product range, keeping smaller discounts, and motivating customers and staff to maintain strong sales in the future.

This project helped to understand the company’s overall sales performance and identify key areas for improvement.
The analysis showed which brands, staff, and cities performed best, and how discounts affect total revenue.
With the right strategy, the company can continue to grow and improve its results.

Thank you for your attention!